

Runp

runp contributors

Version 0.11.1

Overview

Like `docker-compose` but not just for containers.

Today, every non-trivial software project is compound of a number of parts (applications, libraries, services) running together in a lot of environments: virtual machines, Docker containers, physical boxes, cloud...

Runp allows you to specify what and how you want to run and helps you to start the system as a whole, running all needed processes in parallel (like Docker compose, Foreman et simili).

This speeds up the setup of a working environment compared to the usual approaches like custom scripts, documentation (written or spoken), or, more commonly, leaving it to the developer to figure out.

Runp uses a `Runfile`, a sort of a sophisticated `Procfile` that describes the project as a system composed of `units` each of which can run a different type of process such as:

- processes on the physical box
- container processes
- SSH tunnel processes

Moreover the Runfile handles:

- working directory per process
- environment variables per process
- user defined vars per system, defined in the Runfile or passed at runtime as command line arguments
- processes dependency: a process can wait for a given condition to start, e.g. a file appearing or another process being ready
- explicit startup ordering via `depends_on`
- not only *long running* processes but *one shot commands* too
- Windows OS

An example of a `Runfile` :

```
name: Example
description: |
  Sample Runfile to show runp functionalities
units:
  web:
    description: Web app
    # this process is running on host machine
    host:
      command: node app.js
      workdir: backend
```

```
env:
  # inherit PATH from host system to find needed tools (e.g. node)
  PATH: $PATH
await:
  # wait for the DB being available
  resource: tcp4://localhost:5432/
  timeout: 0h0m10s
mail:
  description: Test mail server
  # this process is running in a container
  container:
    image: docker.io/mailhog/mailhog
    ports:
      - "8025:8025"
      - "1025:1025"
db:
  description: Corporate DB
  # This process is reachable through SSH port forwarding
  ssh_tunnel:
    user: user
    auth:
      identity_file: ~/.ssh/id_rsa
    local:
      port: 5432
    jump:
      host: dev.host
      port: 22
    target:
      host: corporate.db
      port: 5432
```

Commands

runp up

Start all units defined in the Runpfile. This is the **default command**: running `runp` with no subcommand is equivalent to `runp up`.

```
runp up
runp up --file ./infra/Runpfile
runp up --var DB_HOST=localhost --var DB_PORT=5432
runp up --key-env RUNP_SECRET_KEY
runp up --dry-run
```

Flags:

Flag	Description
<code>--file / -f</code>	Path to the Runpfile. Overrides the <code>RUNP_FILE</code> environment variable. Default: <code>./Runpfile</code> .
<code>--var / -V</code>	Set a runtime variable declared in the <code>vars:</code> section, e.g. <code>--var DB_HOST=localhost</code> . Can be repeated.
<code>--key-env</code>	Name of the environment variable that holds the decryption key for <code>encrypted_secret</code> fields (recommended).
<code>--key / -k</code>	Decryption key as a plain-text argument. Visible in <code>ps aux</code> and <code>shell history</code> — prefer <code>--key-env</code>.
<code>--dry-run / -n</code>	Print the units that would be started (name, kind, resolved command) without actually starting any process.

runp status (ps)

Show the runtime state of all units.

```
runp status
runp ps
runp status --file ./infra/Runpfile
```

Prints a table with columns NAME, KIND, STATE, and DETAIL. The `ps` alias is provided for convenience.

runp list (ls)

List all units defined in the Runpfile.

```
runp list
runp ls
runp list --output json
runp list --output json | jq '[][.name]'
runp list --file ./infra/Runpfile
```

Flags:

Flag	Description
<code>--output / -o</code>	Output format. <code>table</code> (default) prints an aligned table with NAME, KIND, DESCRIPTION, PRECONDITIONS columns. <code>json</code> emits a JSON array suitable for piping to <code>jq</code> .

runp validate

Validate the Runpfile without starting any process.

```
runp validate
runp validate --file ./infra/Runpfile
```

Exit codes:

- `0` — file is valid
- `1` — validation errors found (missing process type, undefined variable, unknown `depends_on` reference, cycle)
- `2` — file cannot be loaded (not found or invalid YAML)

runp reload

Stop and restart a single unit by name while the rest of the stack keeps running. Run this in a separate shell while `runp up` is active.

```
runp reload web
runp reload api --file ./infra/Runpfile
```

SSH tunnel units are not supported by `reload`.

runp encrypt

Encrypt a secret value for use as `encrypted_secret` in a Runpfile.

```
runp encrypt --key-env RUNP_SECRET_KEY mysecretvalue
echo mysecretvalue | runp encrypt --key-env RUNP_SECRET_KEY
```

```
runp encrypt mysecretvalue          # generates and prints a random key
```

The secret can be supplied as a positional argument or piped via stdin. When neither is provided and stdin is a terminal, the command exits with an error.

WARNING

Avoid `--key` with a literal value — the key will be visible in `ps aux` output and recorded in your shell history. Use `--key-env` instead:

```
export RUNP_SECRET_KEY=my-encryption-key
runp encrypt --key-env RUNP_SECRET_KEY mysecretvalue
```

runp config

Manage user settings stored in `~/.runp/settings.yaml`.

Subcommand	Description
<code>runp config show</code>	Print the full contents of <code>~/.runp/settings.yaml</code> and its path.
<code>runp config get <key></code>	Print the current value of a setting key (returns the default when not explicitly set).
<code>runp config set <key> <value></code>	Write a setting key. Creates <code>~/.runp/settings.yaml</code> if it does not exist.

Example:

```
runp config show
runp config get container_runner
runp config set container_runner podman
```

Supported keys:

Key	Default	Description
<code>container_runner</code>	<code>docker</code>	Container runtime executable used for container units.

Global flags

Flag	Description
<code>--log-level / -l</code>	Log verbosity. Accepted values: <code>trace</code> , <code>debug</code> , <code>info</code> (default), <code>warn</code> , <code>error</code> .
<code>--no-color / -C</code>	Disable colored output. Also honoured via the <code>NO_COLOR</code> environment variable.
<code>--dry-run / -n</code>	Preview units without starting them (equivalent to <code>runc up --dry-run</code>).
<code>--debug / -d</code>	Deprecated. Equivalent to <code>--log-level debug</code> .
<code>--quiet / -q</code>	Deprecated. Equivalent to <code>--log-level warn</code> .

Usage

Run runp

Some commands:

```
runp --help                # generic help
runp help up               # describes the command "up"
runp up                    # run the Runfile in the current directory
(default command)
runp --log-level debug up -f /path/to/runpfile.yaml
runp encrypt --key-env RUNP_KEY secret # encrypt "secret" using the key from
$RUNP_KEY
runp ls -f /path/to/runpfile.yaml     # list units in Runfile
runp validate -f /path/to/runpfile.yaml # validate Runfile without starting
runp status                           # show runtime status of all units
```

Selecting the Runfile

The Runfile path is resolved in the following order of precedence:

1. `--file / -f` flag
2. `RUNP_FILE` environment variable
3. `./Runfile` (default)

Example using the environment variable:

```
export RUNP_FILE=/path/to/my/Runfile
runp up
runp validate
```

Settings

Runp looks for a settings file in `~/.runp/settings.yaml`.

If the file does not exist or is empty/invalid, Runp starts using these defaults:

```
container_runner: docker
```

Use `runp config` to read or update settings without editing the file directly:

```
runp config set container_runner podman
runp config get container_runner
runp config show
```

Preconditions

Preconditions can be defined at two levels: at the root of the **Runfile** or within a specific **unit**.

- **Root-level preconditions:** These are checked before any unit is executed. If any of these preconditions are not met, **runp** will exit and no units will be started. This is useful for ensuring that the environment is correctly set up before attempting to run any part of the system.
- **Unit-level preconditions:** These are checked before the specific unit is executed. If the preconditions for a unit are not met, that unit is skipped, but other units in the **Runfile** will still be executed.

Available preconditions:

- **os:** This precondition checks if the operating system matches one of the specified values.
- **runp:** This precondition checks if the **runp** version satisfies the specified constraint.
- **hosts:** This precondition checks if **/etc/hosts** contains the given mapping.
- **env_vars:** This precondition checks environment variables.

os

The **os** precondition checks if the operating system is one of the specified **inclusions**.

Example:

```
preconditions:
  os:
    inclusions:
      - linux
      - darwin
```

In this example, the execution will only proceed if the operating system is Linux or macOS.

runp

The **runp** precondition checks if the **runp** version satisfies the specified constraint. The available operators are: **LessThan**, **LessThanOrEqualTo**, **Equal**, **GreaterThanOrEqualTo**, **GreaterThan**.

Example:

```
preconditions:
  runp:
    operator: GreaterThanOrEqualTo
    version: 0.10.0
```

This will ensure that **runp** version is at least **0.10.0**.

hosts

The `hosts` precondition checks for the presence of specific entries in the `/etc/hosts` file.

Example:

```
preconditions:
  hosts:
    - name: my.special.host
      ip: 127.0.0.1
```

This precondition is satisfied if `my.special.host` resolves to `127.0.0.1`.

`env_vars`

The `env_vars` precondition checks for the presence or value of environment variables. The available conditions are `is_set`, `is_not_set`, `equals`, `not_equals`, `contains`, `not_contains`.

Example of a root-level precondition:

```
preconditions:
  env_vars:
    - name: DATABASE_URL
      condition: is_set
```

In this case, `runp` will exit if the `DATABASE_URL` environment variable is not set.

Example of a unit-level precondition:

```
units:
  my-service:
    preconditions:
      env_vars:
        - name: MY_SERVICE_ENABLED
          condition: equals
          value: "true"
    host:
      command: "echo 'My service is enabled'"
```

In this example, the `my-service` unit will only be executed if the `MY_SERVICE_ENABLED` environment variable is set to `"true"`.

Runfile composition with `include`:

A Runfile can include other Runfiles using the `include` key. Each entry is a path **relative to the directory of the file that declares it**. All units from included files are merged into the root Runfile before startup.

```
name: My Project
```

```
include:
- infra/Runpfile      # defines units: db, redis
- services/Runpfile   # defines units: api, worker
units:
  proxy:
    host:
      command: nginx -g "daemon off;"
```

After loading, `runp up` starts all units from all included files together.

Includes can be nested to any depth.

WARNING

Duplicate unit names across any two files are an error — loading fails with:

```
duplicate unit identifier: <name>
```

Circular includes are detected and reported with the full import chain:

```
circular dependency detected: a.yml → b.yml → c.yml → b.yml
```

Startup ordering with `depends_on`

By default all units start concurrently. Use `depends_on` to declare explicit ordering when one unit must be fully running before another begins.

```
units:
  db:
    container:
      image: postgres:15
      ports: ["5432:5432"]
  redis:
    container:
      image: redis:7
  api:
    depends_on: [db, redis]
    host:
      command: ./api-server
  worker:
    depends_on: [api]
    host:
      command: ./worker
```

Runp groups units into **dependency layers** and starts each layer concurrently, waiting for it to complete before starting the next:

- Layer 0: `db`, `redis` (no dependencies)

- Layer 1: `api` (depends on `db` and `redis`)
- Layer 2: `worker` (depends on `api`)

`runp validate` reports unknown references and circular dependencies as errors.

Units that are skipped due to unsatisfied preconditions are excluded from the graph; their dependents are treated as if the dependency is already satisfied.

Stop timeout

For every unit, you can specify a timeout to wait for the process to stop gracefully. If the process does not stop within the timeout, it will be killed. The default value is 5 seconds.

```
units:
  zookeeper:
    description: Zookeeper
    stop_timeout: 0h0m15s
    container:
      name: zookeeper
      image: strimzi/kafka:0.19.0-kafka-2.5.0
      command: "bin/zookeeper-server-start.sh config/zookeeper.properties"
      ports:
        - "2181:2181"
      env:
        LOG_DIR: /tmp/logs
```

App waiting for another resource

A unit can wait for a resource to be available before starting. The `await` block can be used to specify a `resource` and a `timeout`.

If the `resource` is not specified, the unit will wait for the given `timeout`.

```
units:
  app:
    description: app
    host:
      command: echo hi
      env:
        PATH: $PATH
      await:
        timeout: 0h0m03s
```

A backend app running on host waiting for a DB running in a container to be available:

```
units:
  be:
    description: Backend app
```

```

host:
  command: mvn clean compile quarkus:dev
  workdir: backend
  env:
    # inherit PATH from host system to find mvn and java
    PATH: $PATH
  await:
    resource: tcp4://localhost:5432/
    timeout: 0h0m10s
db:
  description: Database
  container:
    image: docker.io/postgres:alpine
  ports:
    - "5432:5432"
  env:
    POSTGRES_PASSWORD: pass
    POSTGRES_USER: user
    POSTGRES_DB: dbname

```

Wait for a file to be available:

```

units:
  await:
    description: command
  host:
    command: env
    await:
      resource: file:///tmp/some_file.txt
      timeout: 0h0m9s
  env:
    MYHOME: ${HOME}

```

Wait for an HTTP resource to be available:

```

units:
  run:
    description: docker from shell
  host:
    command: docker run -t --rm hello-world
    workdir: /tmp
    await:
      resource: http://localhost:8000
      timeout: 0h0m25s
  hello-web:
    description: crccheck/hello-world
    container:
      image: crccheck/hello-world
    ports:

```

```
- "8000:8000"
```

Unit readiness conditions

By default, runp considers a unit "done" (and starts its dependents) when its process exits. For long-running services this is undesirable — dependents should start as soon as the service is up, not when it eventually stops.

The `ready` block configures a readiness condition that tells runp when a unit is ready for its dependents to proceed. Three condition types are supported; at most one should be set per unit. If multiple fields are set, `output` takes priority, then `cmd`, then `delay`.

ready.delay — wait a fixed duration after the process starts:

```
units:
  db:
    description: Database (ready after 3s warm-up)
    ready:
      delay: 3s
      timeout: 30s # how long to wait before giving up (default: 30s)
    host:
      command: postgres -D /var/lib/pgsql/data
  api:
    description: App server (starts after db is ready)
    depends_on: [db]
    host:
      command: node server.js
```

ready.output — match a substring in the process stdout/stderr:

```
units:
  api:
    description: App server (ready when "listening" appears in output)
    ready:
      output: "listening"
      timeout: 30s
    host:
      command: node server.js
  worker:
    description: Background worker (starts after api is ready)
    depends_on: [api]
    host:
      command: node worker.js
```

ready.cmd — poll a shell command every second; ready when it exits 0:

```
units:
```

```
api:
  description: App server (ready when health endpoint responds)
  ready:
    cmd: "curl -sf http://localhost:8080/health"
    timeout: 60s
  host:
    command: ./api-server
consumer:
  description: Consumer (starts after api health check passes)
  depends_on: [api]
  host:
    command: ./consumer
```

NOTE

`ready` and `await` serve complementary roles:

- `await` waits for an **external** resource before **this** unit's process starts (e.g. "wait for the DB port to be open before launching the app").
- `ready` determines when **this** unit signals to its **dependents** that they may proceed (e.g. "dependents start when this service prints 'listening'").

Both can be used together on the same unit.

Containers

You can set the container engine using the settings file (key: `container_runner`).

Example:

```
container_runner: /path/to/podman
```

WARNING

Only Docker and Podman (as they use the same command line flags) are supported.

Containers can talk to each other thorough a Docker network (`runp-network`).

The container name (the host name exposed to other containers) is set to `runp-${UNIT NAME}` or to the field `name`.

This Runfile starts Wordpress and MySql:

```
name: Wordpress Runfile
description: Runfile to run Wordpress and MySql
units:
  db:
    container:
      name: db
      image: docker.io/mysql:5.7
      ports:
```

```

    - "3306:3306"
  env:
    MYSQL_ROOT_PASSWORD: somewordpress
    MYSQL_DATABASE: wordpress
    MYSQL_USER: wordpress
    MYSQL_PASSWORD: wordpress
wordpress:
  container:
    image: docker.io/wordpress:latest
  ports:
    - "8000:80"
  env:
    WORDPRESS_DB_HOST: db:3306
    WORDPRESS_DB_USER: wordpress
    WORDPRESS_DB_PASSWORD: wordpress
    WORDPRESS_DB_NAME: wordpress
  await:
    resource: tcp4://localhost:3306/
    timeout: 0h0m20s

```

Use containers volumes

Run containers and volumes (example is from the book Docker in action - Manning):

```

name: Containers Runfile
description: This is Runfile showing Docker volumes
units:
  fowler:
    description: The Fowler collection
    container:
      image: docker.io/alpine:3.12
      skip_rm: true
      mounts:
        - "type=volume,dst=/library/PoEAA"
        - "type=bind,src=/tmp,dst=/library/DSL"
      command: |
        echo "Fowler collection created"
  knuth:
    description: The Knuth collection
    container:
      image: docker.io/alpine:3.12
      skip_rm: true
      mounts:
        - "type=volume,dst=/library/TAoCP.vol1"
        - "type=volume,dst=/library/TAoCP.vol2"
        - "type=volume,dst=/library/TAoCP.vol3"
      command: |
        echo "Knuth collection created"
  reader:
    description: The avid reader

```

```

container:
  image: alpine:3.12
  volumes_from:
    - fowler
    - knuth
  command: |
    ls -l /library/
  await:
    timeout: 0h0m3s

```

On Windows

Windows is supported:

```

name: Test Runfile
description: This is Runfile
units:
  await:
    description: read environment variables
    host:
      command: set
      env:
        # in env block variables have always the unix notation
        MYHOME: ${HOME}
  echo:
    description: echo the value of %OS% env var
    host:
      # when used in command, env vars have the specific OS notation
      command: echo %OS%
  infiniteloop:
    description: infinite loop
    host:
      # this script is in examples/ directory
      executable: infinite.cmd
      workdir: examples

```

Some programs, especially on Windows, implicitly use OS specific environment variables.

If you run into strange problems try adding these to the env block:

```

# Windows env vars
SystemRoot: ${SystemRoot}
ALLUSERSPROFILE: ${ALLUSERSPROFILE}
APPDATA: ${APPDATA}
CommonProgramFiles: ${CommonProgramFiles}
CommonProgramW6432: ${CommonProgramW6432}
ComSpec: ${ComSpec}
DriverData: ${DriverData}
HOMEDRIVE: ${HOMEDRIVE}

```

```

HOMEPATH: ${HOMEPATH}
LOCALAPPDATA: ${LOCALAPPDATA}
OS: ${OS}
PATHEXT: ${PATHEXT}
ProgramData: ${ProgramData}
ProgramFiles: ${ProgramFiles}
ProgramW6432: ${ProgramW6432}
PSModulePath: ${PSModulePath}
PUBLIC: ${PUBLIC}
SESSIONNAME: ${SESSIONNAME}
SystemDrive: ${SystemDrive}
TEMP: ${TEMP}
TMP: ${TMP}
USERNAME: ${USERNAME}
USERPROFILE: ${USERPROFILE}
windir: ${windir}

```

Specify a shell for host processes

For host processes, you can specify a shell to execute the command. By default, `runp` uses the default shell of the operating system.

```

units:
  bash-process:
    description: a process that runs with bash
    host:
      command: "echo $BASH_VERSION"
      shell:
        path: /bin/bash
        args:
          - -c

```

Run a different command on different operative systems

Inclusions are compared to `runtime.GOOS`:

```

units:
  win:
    description: Windows unit
    preconditions:
      os:
        # this unit is ran when os is windows
      inclusions:
        - windows
    host:
      command: dir {{vars runp_root}}
  unix:
    description: Nix unit
    preconditions:

```

```

os:
  # this unit will be ran when os is linux or darwin
  inclusions:
    - linux
    - darwin
host:
  command: ls -al {{vars runp_root}}

```

SSH tunnel to reach a remote LDAP

A backend app running on host using LDAP on remote server available using SSH tunneling.

SSH tunnel can use three auth methods:

- `identity_file`: the path to the private key, ie `~/.ssh/id_rsa`
- `secret`: the SSH server password in plain text
- `encrypted_secret`: the SSH server password encrypted and in base 64 (you can create it using `runp encrypt`)

```

units:
  be:
    description: Backend app
    host:
      command: mybackendapp
      workdir: backend
  ldap:
    description: LDAP
    ssh_tunnel:
      user: runp
      auth:
        #identity_file: ~/tmp/runpssh/ssh/runp
        #secret: "plain text secret"
        encrypted_secret: "NsM1hcAy/L2TfACgfzbhYyb9j5a2ySYcARFDKkv7HTk="
      local:
        # localhost is the default
        port: 389
      jump:
        host: sshserver
        port: 22
      target:
        host: ldapserver
        port: 389

```

WARNING

Setting `insecure_ignore_host_key: true` on an SSH tunnel unit disables host key verification and makes the connection vulnerable to man-in-the-middle attacks. Runp always prints a warning to stderr when this option is active, regardless of the `--log-level` setting. Use it only in controlled environments and never in production.

```
ssh_tunnel:
  insecure_ignore_host_key: true  # WARNING: MITM risk
  ...
```

Use secrets

SSH tunnel process allows user to use secrets to specify the password.

To create the encrypted secret:

```
export RUNP_SECRET_KEY=thekey
runp encrypt --key-env RUNP_SECRET_KEY SECRET
```

The above command will encrypt the string **SECRET** using the key stored in **\$RUNP_SECRET_KEY**.

You can also pipe the secret from stdin to avoid it appearing in your shell history:

NOTE

```
echo SECRET | runp encrypt --key-env RUNP_SECRET_KEY
```

If no key is provided at all, **runp encrypt** generates a random key and prints it alongside the encrypted value.

To run a Runpfile containing an **encrypted_secret** you have to pass the key to the **up** command (the key must coincide with the one used to encrypt).

Use **--key-env** to pass the key through an environment variable (recommended):

```
export RUNP_SECRET_KEY=thekey
runp up --key-env RUNP_SECRET_KEY
```

WARNING

The **--key** / **-k** flag passes the key as a plain-text command-line argument. It will be visible in **ps aux** output and recorded in your shell history. Always prefer **--key-env**.

Use environment variables

A one-shot command using custom environment variables:

```
env3:
  description: echo command
  host:
    command: echo ${MYHOME}
  workdir: ..
  env:
```

```
MYHOME: ${HOME}
```

User defined variables

Use runtime vars:

```
vars:
  foo: FOO_DEFAULT_VALUE
units:
  vars-test-unit:
    description: echo a user defined var
    host:
      command: echo __{{vars foo}}__
```

The var `foo` will have value `FOO_DEFAULT_VALUE` or can be set from command line:

```
$ bin/runp --log-level debug up -f examples/Runfile-vars.yml --var foo=bar
```

Implicit variables

Runp adds to the context some variables:

- `runp_workdir`: user current working directory as absolute path
- `runp_root`: directory parent of the Runfile as absolute path
- `runp_file_separator`: OS file separator (/ on unix, \ on windows)

Usage:

```
units:
  vars:
    description: echo implicit vars from Runp
    host:
      command: "echo runp_workdir={{vars runp_workdir}} runp_root={{vars runp_root}}"
```

Disabling color output

To have plain, non-colored text output set the environment variable `NO_COLOR`:

```
NO_COLOR=1 ./bin/runp --log-level debug up -f examples/Runfile-many-units.yml
```

or use the option `--no-color`:

```
./bin/runp --log-level debug --no-color up -f examples/Runfile-many-units.yml
```

Runpfile Runp version

A unit can require a constraint on the Runp version.

This unit requires runp version greater the 0.5.0:

```
units:
  test1:
    description: test unit
    preconditions:
      runp:
        operator: GreaterThan
        version: 0.5.0
    host:
      command: env
      workdir: ${HOME}
```

The available operators are:

- LessThan
- LessThanOrEqual
- Equal
- GreaterThanOrEqual
- GreaterThan

License

[Apache License 2.0](#) - Copyright © 2020-TODAY runp contributors.